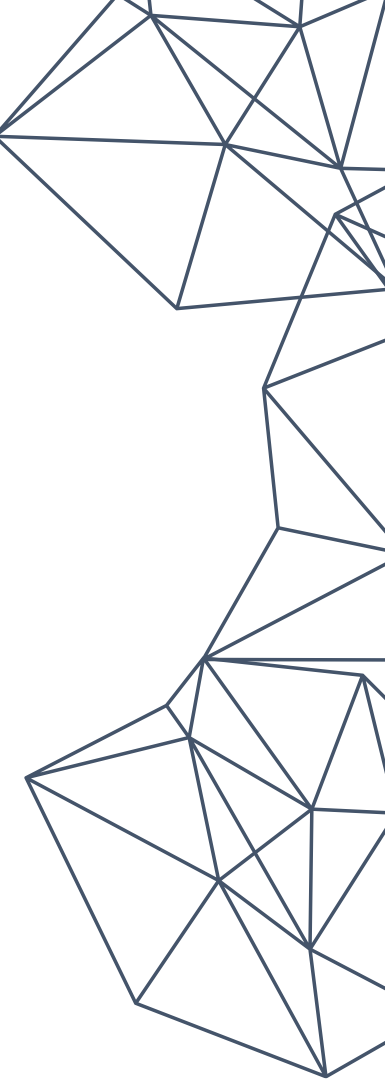




Mobile Application Security & Penetration Testing

Android, iOS, WhatsApp Risk Paths



Mobile App Penetration Testing Methodology

Phase	What We Test	Output
Scope & Setup	App build, test accounts, roles, APIs, device type	Rules of engagement + lab plan
Static Review	APK/IPA structure, secrets, permissions, configs	Hardcoded keys, risky flags, exposed logic
Dynamic Testing	Traffic, auth, storage, runtime behavior, APIs	Validated findings with evidence
Reporting	Business impact, exploit path, screenshots, fixes	Risk-rated report + retest plan



Android App Penetration Testing Focus

Static Review

- Decompile APK/AAB and inspect AndroidManifest.xml.
- Review permissions, exported activities/services/receivers.
- Look for hardcoded API keys, URLs, secrets and debug flags.
- Check insecure local storage: Shared Preferences, SQLite, files and logs.

Dynamic Review

- Proxy app traffic through Burp and test APIs.
- Test certificate pinning behavior and safe bypass in lab.
- Use emulator/rooted test device for Frida/Objection runtime checks.
- Validate auth, session handling, IDOR and business logic issues.

Android testing is flexible because analysts can instrument apps and inspect runtime behavior more easily.



iOS App Penetration Testing Focus

- Review IPA structure, Info.plist, entitlements and embedded frameworks.
- Check Keychain usage, local files, app groups, URL schemes and pasteboard exposure.
- Inspect ATS settings, jailbreak checks, debug symbols and third-party SDKs.
- Use a supervised test device or jailbroken lab device where allowed.
- Proxy traffic through Burp and validate.

IPA / Plist
Config +
entitlements

Keychain
Storage review

Proxy
API traffic

Runtime
Frida / Objection

Jailbreak
Lab only



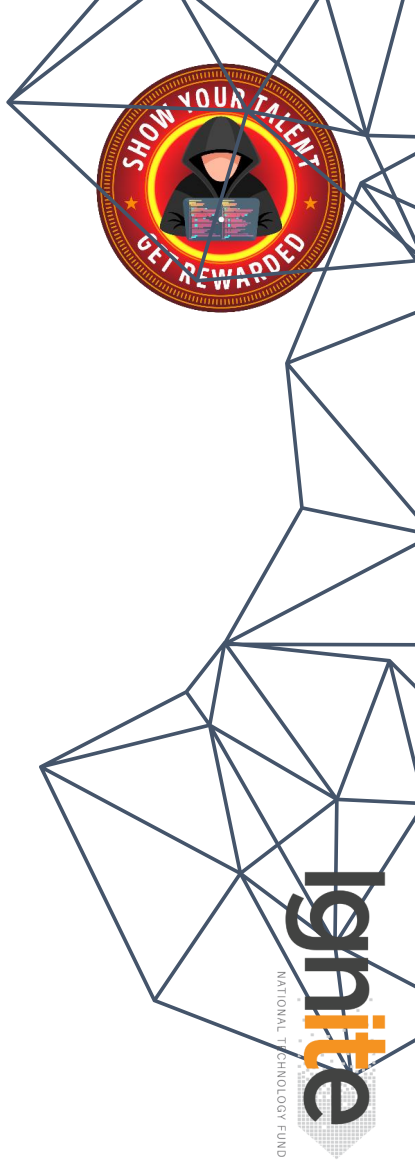
iOS: Inside the IPA File

An IPA is just a signed zip file containing the app's code, resources, and settings (Info.plist)

Info.plist misconfigurations: settings like custom URL schemes or network exceptions left too open.

ATS bypass: a setting (NSAllowsArbitraryLoads) that turns off Apple's secure connection requirement — sometimes left on by mistake in production.

Keychain protection classes: control when stored data can be accessed (e.g., only when unlocked, or always) — picking the wrong one weakens security.



iOS Logic Flaws: URL Schemes & Deeplinks

- Apps can register a custom URL scheme (like myapp://) to open from links — but nothing stops another app from registering the same one.
- If a malicious app claims the same scheme, it can intercept data meant for the real app (including login tokens).
- Deeplinks can also be abused directly — passing unexpected parameters to skip login or access another user's data.

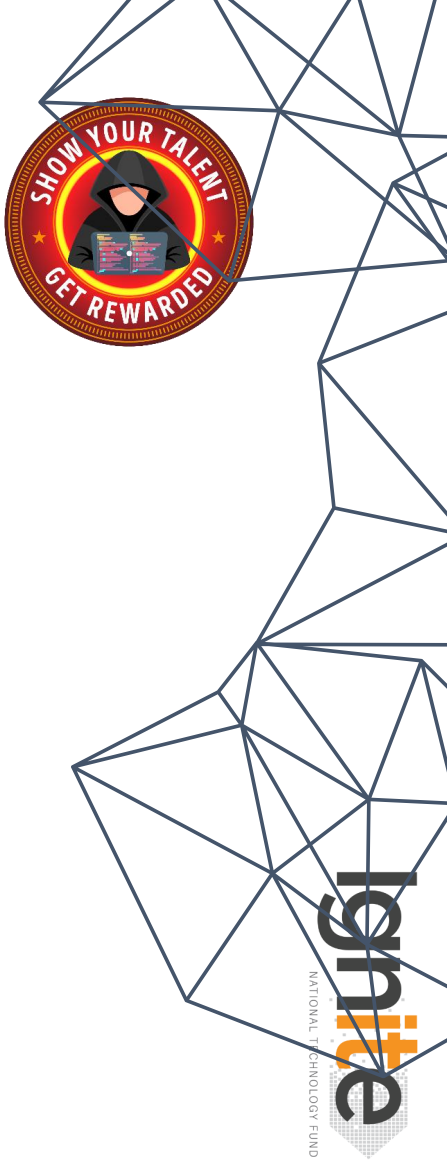
Real example: an attacker's app registers the same scheme as a banking app's OAuth callback, and captures the login token.



Manipulating the App While It Runs

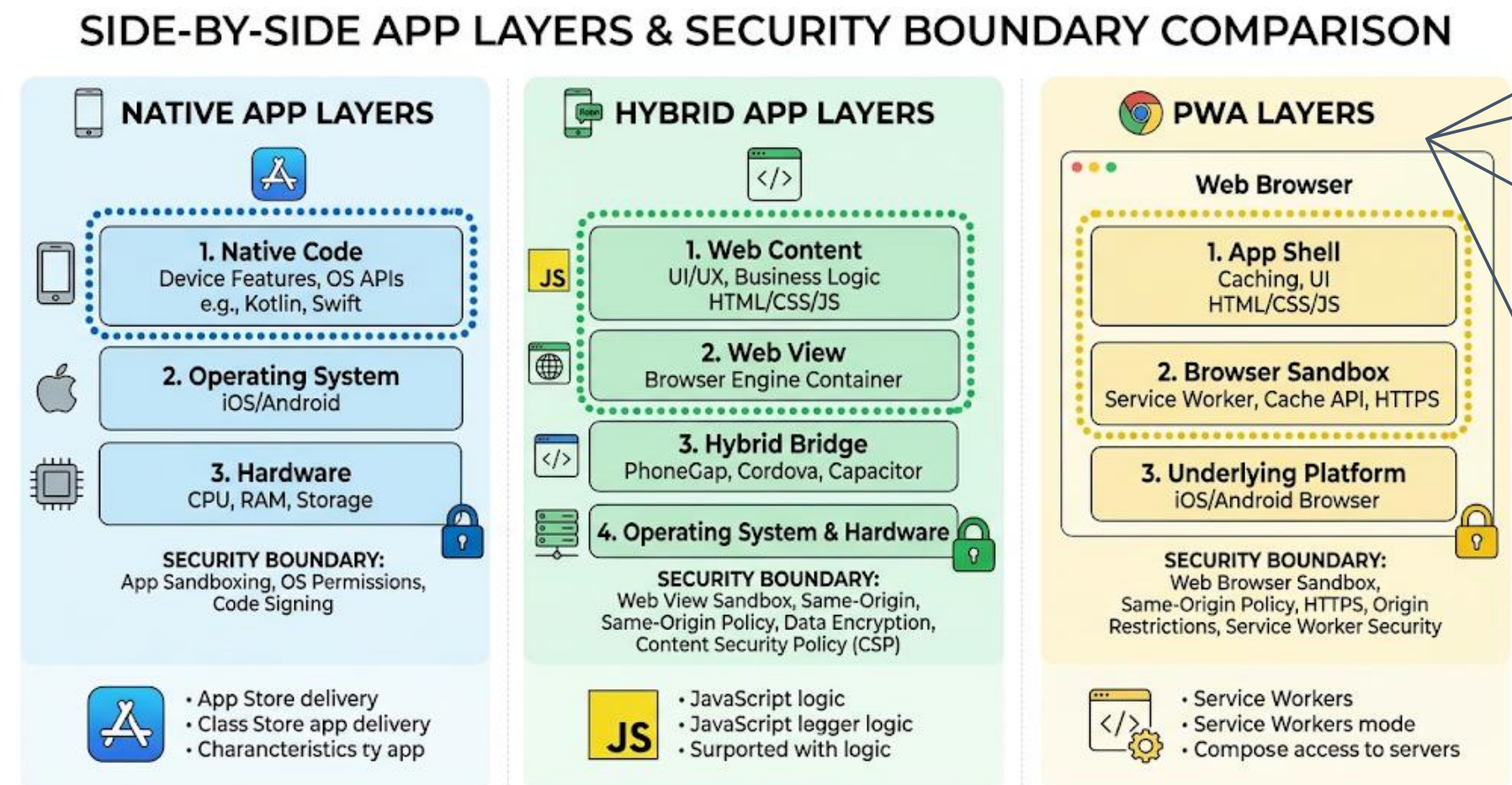
- **Method swizzling (iOS):** swapping out one function for another at runtime, so the app runs different code than intended.
- **ART hooking (Android):** intercepting how the Android Runtime executes app code, to change its behavior.
- **Frida gadget injection:** embedding Frida directly into the app file (APK/IPA) so it works even without root or jailbreak.

Why testers use this: to bypass root/jailbreak detection, force specific code paths, or read sensitive data straight from memory.



App Architecture & Why It Matters for Security

- **Native apps (Swift/Kotlin):** talk to the OS directly, best sandboxing, but the code is easiest to reverse-engineer.
- **Hybrid apps (React Native, Flutter, Cordova):** app logic often sits in a plain JS bundle or Dart file that isn't encrypted — attackers can read it.
- **PWAs (Progressive Web App):** run inside the browser, so they don't get Keychain/Keystore protection; cached data in the service worker can leak.
- Why it matters: the tools you use to test an app depend on its architecture (e.g., unpacking a JS bundle vs decompiling native code).



Common Mobile App Vulnerabilities

- Insecure data storage: tokens, PII, documents or screenshots saved locally.
- Weak authentication: long-lived tokens, no device binding, missing MFA for sensitive actions.
- Broken authorization: user can access another user's records through API parameter changes.
- Insecure communication: weak TLS settings, no pinning where needed, leaking data to third-party SDKs.
- Reverse engineering exposure: hardcoded secrets, debug endpoints and sensitive logic in client code.

Storage
Tokens + PII

Auth
MFA + sessions

APIs
IDOR + access control



Bypassing Certificate Pinning

Certificate pinning prevents the app from trusting interception certificates that do not match the expected certificate/public key, which can block HTTPS inspection through Burp Suite or similar proxies.

- **Frida:** lets you hook into the app while it's running and patch out the pinning check
- **SSL Kill Switch (iOS, jailbroken devices):** disables pinning system-wide with one tool
- **Objection:** built on Frida, does the same thing with a single command — no scripting needed

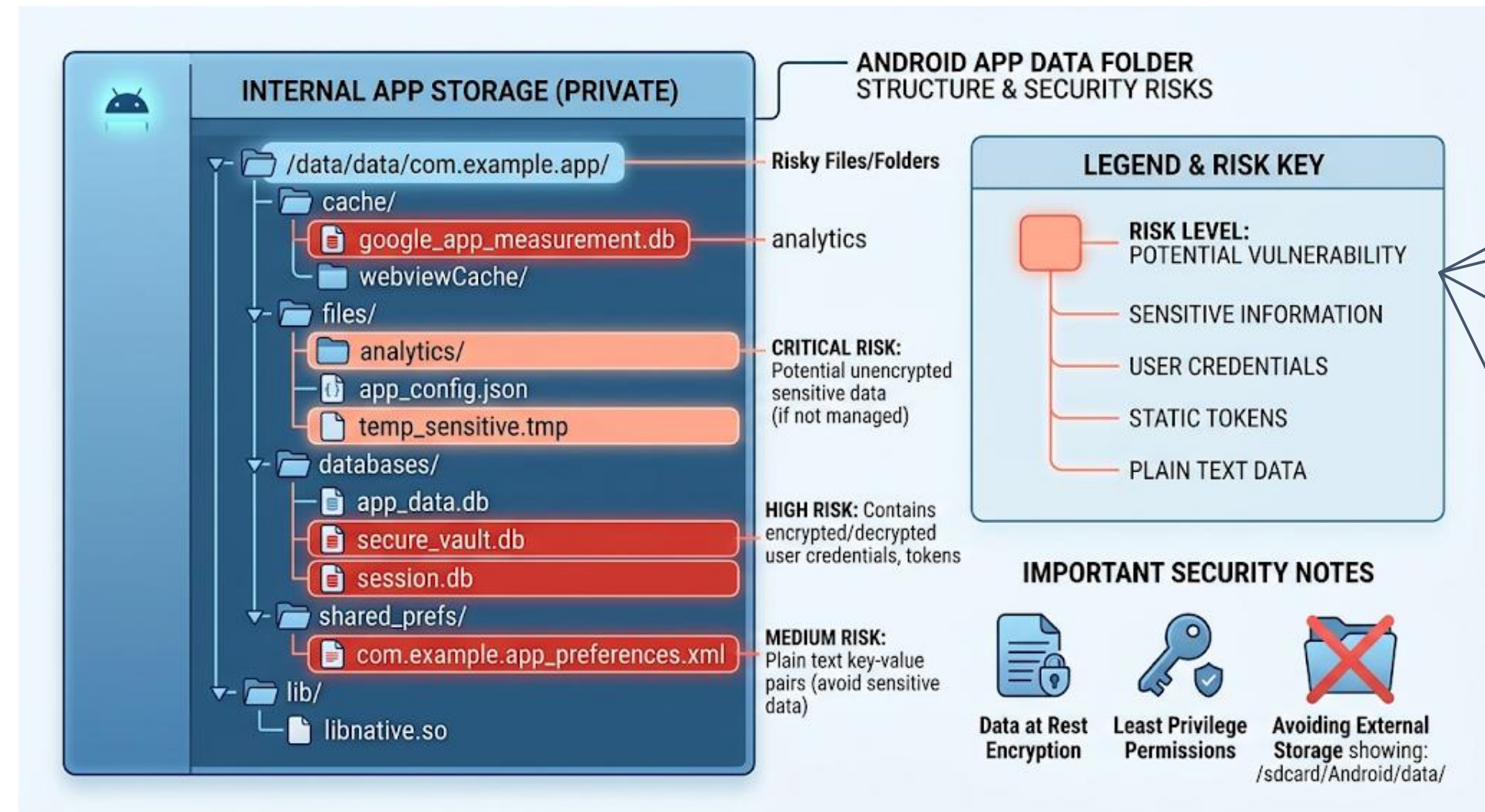
Note: only done on test devices/lab apps with permission



Insecure Data Storage

- **Plaintext SQLite databases:** many apps store data in a local database with no encryption — anyone with file access can read it
- **SharedPreferences (Android) / UserDefaults (iOS):** simple key-value storage, often used to save tokens or personal data without protection
- **World-readable files:** files saved with permissions that let other apps on the device read them

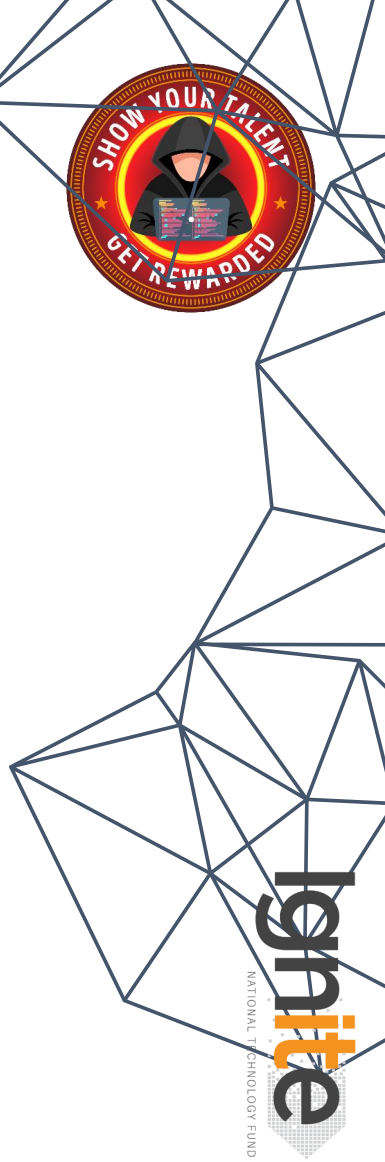
How testers find this: pull the app's data folder and search it for tokens, passwords, or personal info



Where Tokens and Secrets Go Wrong

- **Hardcoded API keys:** secrets left directly in the app's code, visible once decompiled
- **Tokens in logs:** session tokens accidentally printed to the device log (Logcat on Android, console on iOS)
- **Keychain misuse (iOS):** token saved with the wrong protection setting, so it stays readable even when the phone is locked
- **Keystore misuse (Android):** keys stored without hardware protection, making them easier to extract

Fix: use hardware-backed storage (Secure Enclave / StrongBox) and the strictest access settings



Bypassing Client-Side Checks

Any check done only in the app (login limits, paywalls, feature locks) can be changed by the user — it's not real security

Frida/Objection: Lets testers hook the function doing the check and force it to always pass

Common examples: skipping a paywall, defeating root/jailbreak detection, unlocking hidden features

Frida CodeShare



Log In

Search

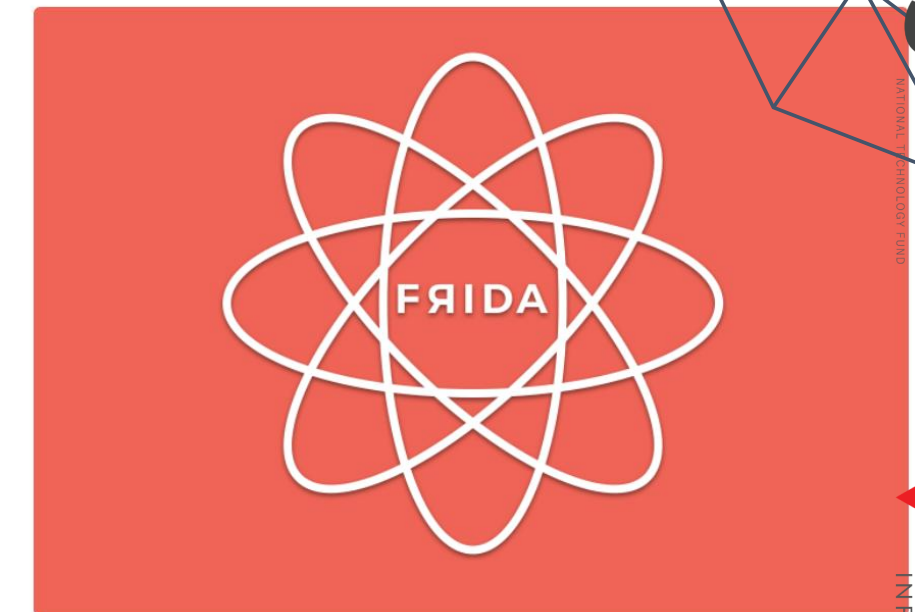
Unleash the power of Frida.

"If I have seen further, it is by standing on the shoulders of giants."
-Sir Issac Newton

The Frida CodeShare project is comprised of developers from around the world working together with one goal - push [Frida](#) to its limits in new and innovative ways.

Frida has [amazing potential](#), but needed a better forum to share ideas, so we've put together CodeShare to help stoke innovation and push Frida to its highest potential!

[BROWSE CODE](#)

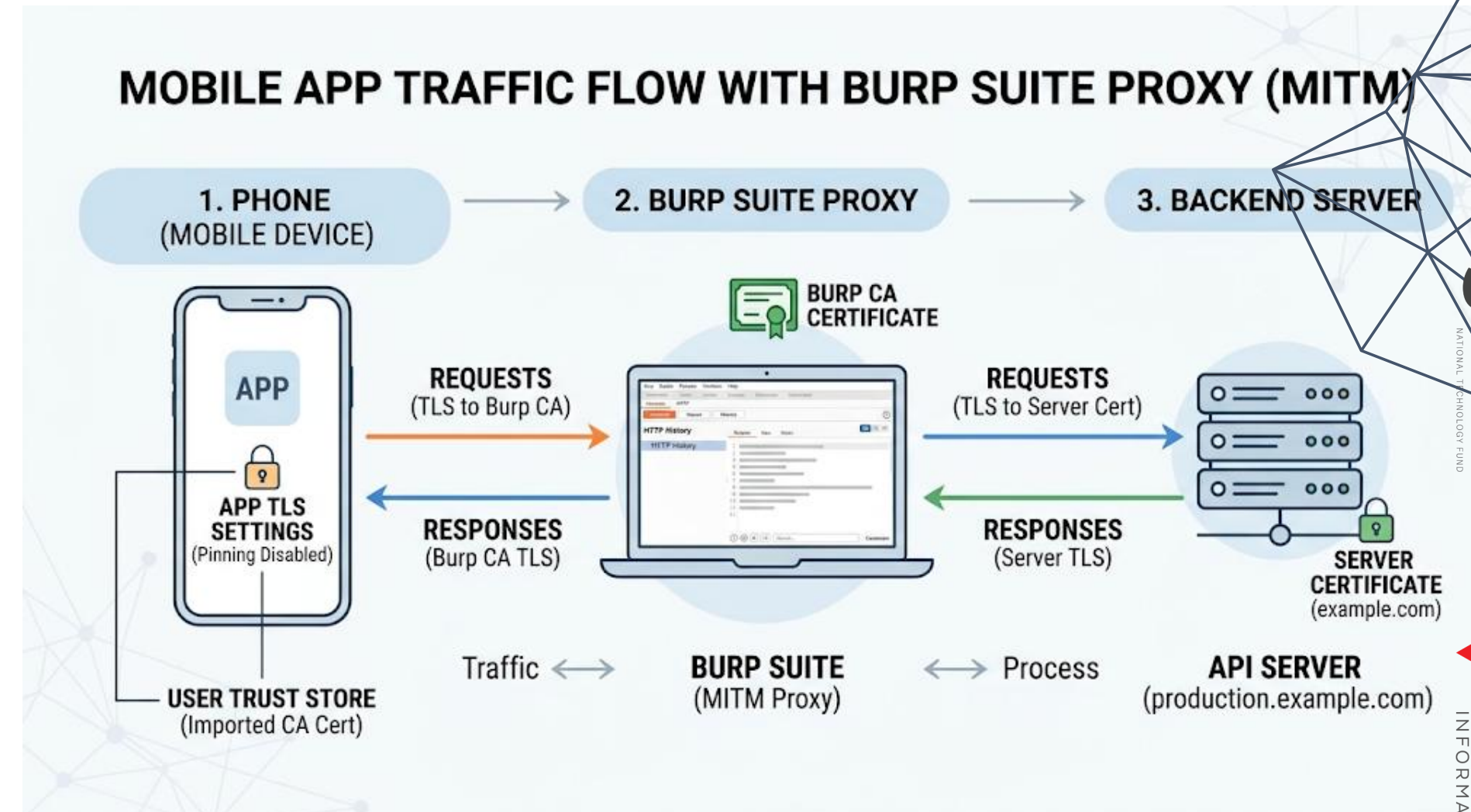


ignite
NATIONAL TECHNOLOGY FOUND

TRILLIUM
INFORMATION SECURITY SYSTEMS

Intercepting Mobile Traffic (MitM) with Burp Suite

- **Android setup:** install Burp's certificate on the device, and work around Android's network security rules (especially Android 7+); rooted devices may need a system-level cert install
- **iOS setup:** install Burp's certificate as a profile, then manually trust it in Settings
- **Common blocker:** certificate pinning stops traffic from showing up in Burp at all — this is why Slide C's bypass techniques come first

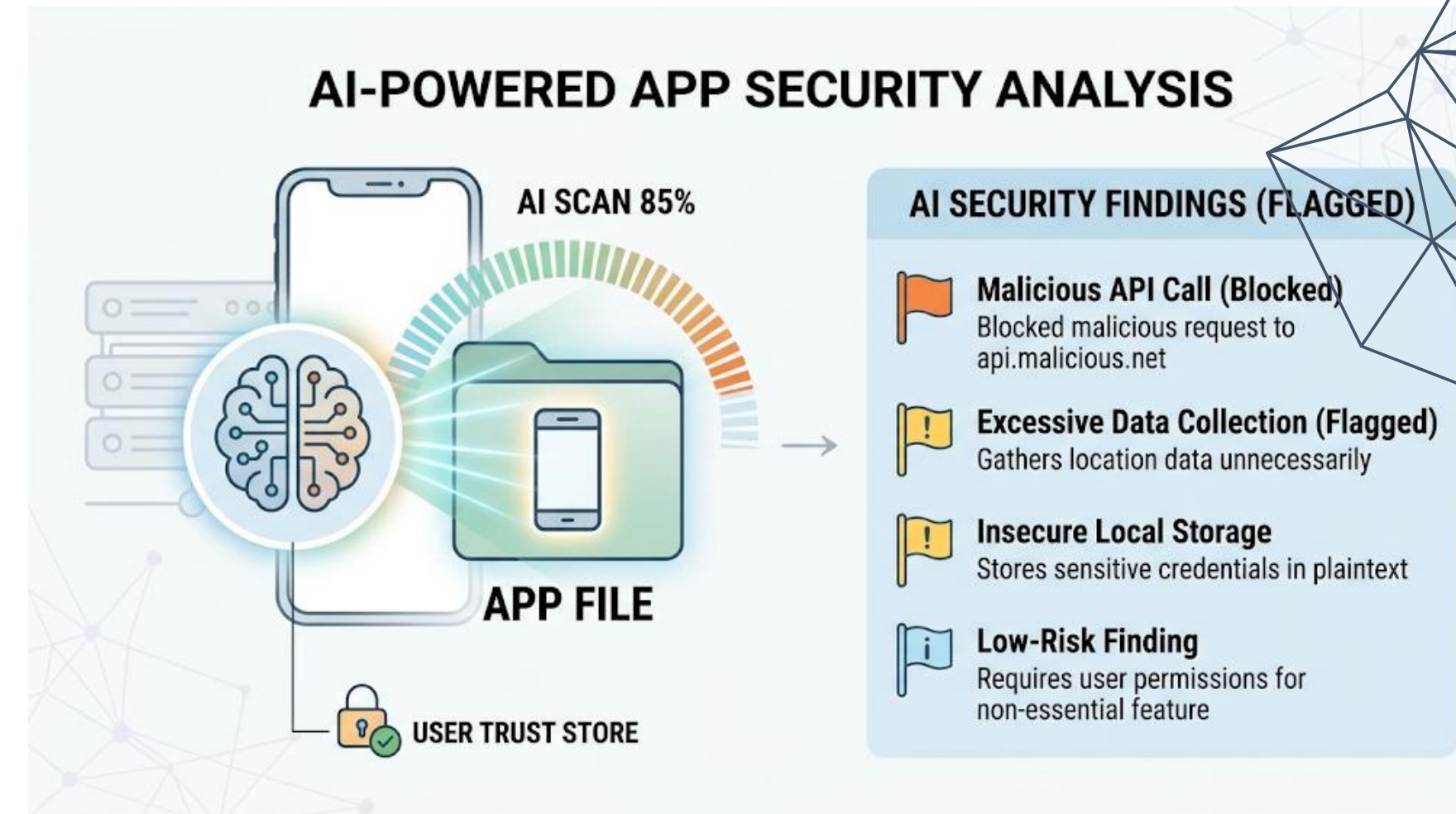


AI in Mobile App Security

AI-assisted static analysis: an LLM scans an app's code (APK/IPA) and automatically flags risky API calls, hardcoded secrets, and weak settings

AI-assisted dynamic analysis: tools like MobSF now use machine learning to score how risky an app's runtime behavior is

AI-generated Frida scripts: AI can draft bypass scripts for common scenarios (like pinning or root detection) — but a human should always review them before use



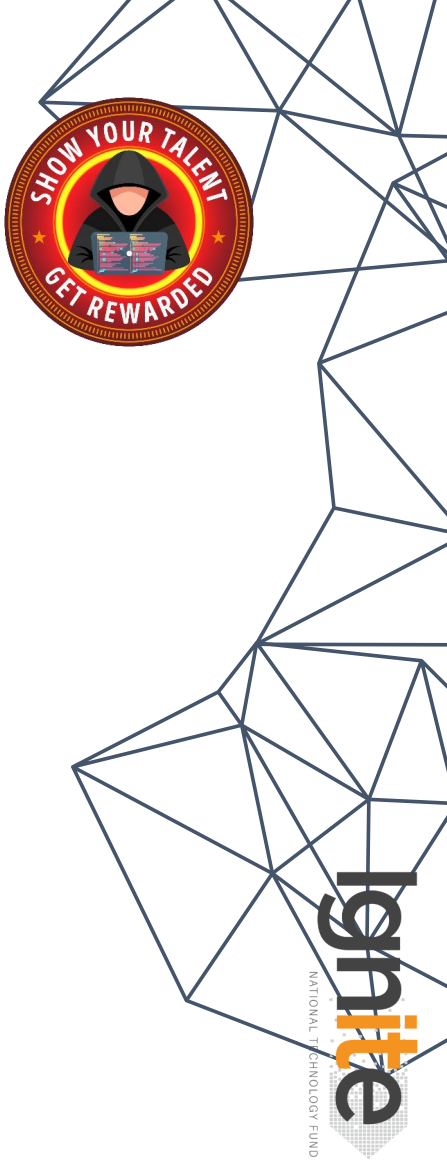
Case Study: AI-Assisted Mobile Pentest in Action

Scenario: Tester given an unfamiliar banking APK with a tight deadline.

- 1. Static analysis:** LLM scans the decompiled code, flags a hardcoded API key and an exported activity with no permission check in minutes instead of hours
- 2. Dynamic analysis:** MobSF's ML scoring highlights the app's runtime network calls as high-risk due to a weak TLS config
- 3. Exploitation:** AI drafts a starting Frida script to bypass certificate pinning; tester reviews, adjusts, and runs it

Result: Manual triage time cut significantly — but every AI finding and script was human-verified before being used or reported

Takeaway: AI speeds up discovery, it doesn't replace judgment — false positives and unsafe scripts still happen



Android vs iOS: Penetration Testing Differences

Area	Android	iOS
App package	APK/AAB; easier to decompile and modify	IPA; signing and device trust are stricter
Runtime testing	Rooted emulator/device commonly used	Jailbroken or supervised test device often needed
Storage focus	Shared Preferences, SQLite, files, logs	Keychain, app container, pasteboard, app groups
Common challenge	Fragmentation, OEM differences, rooted detection	Closed ecosystem, signing, jailbreak restrictions

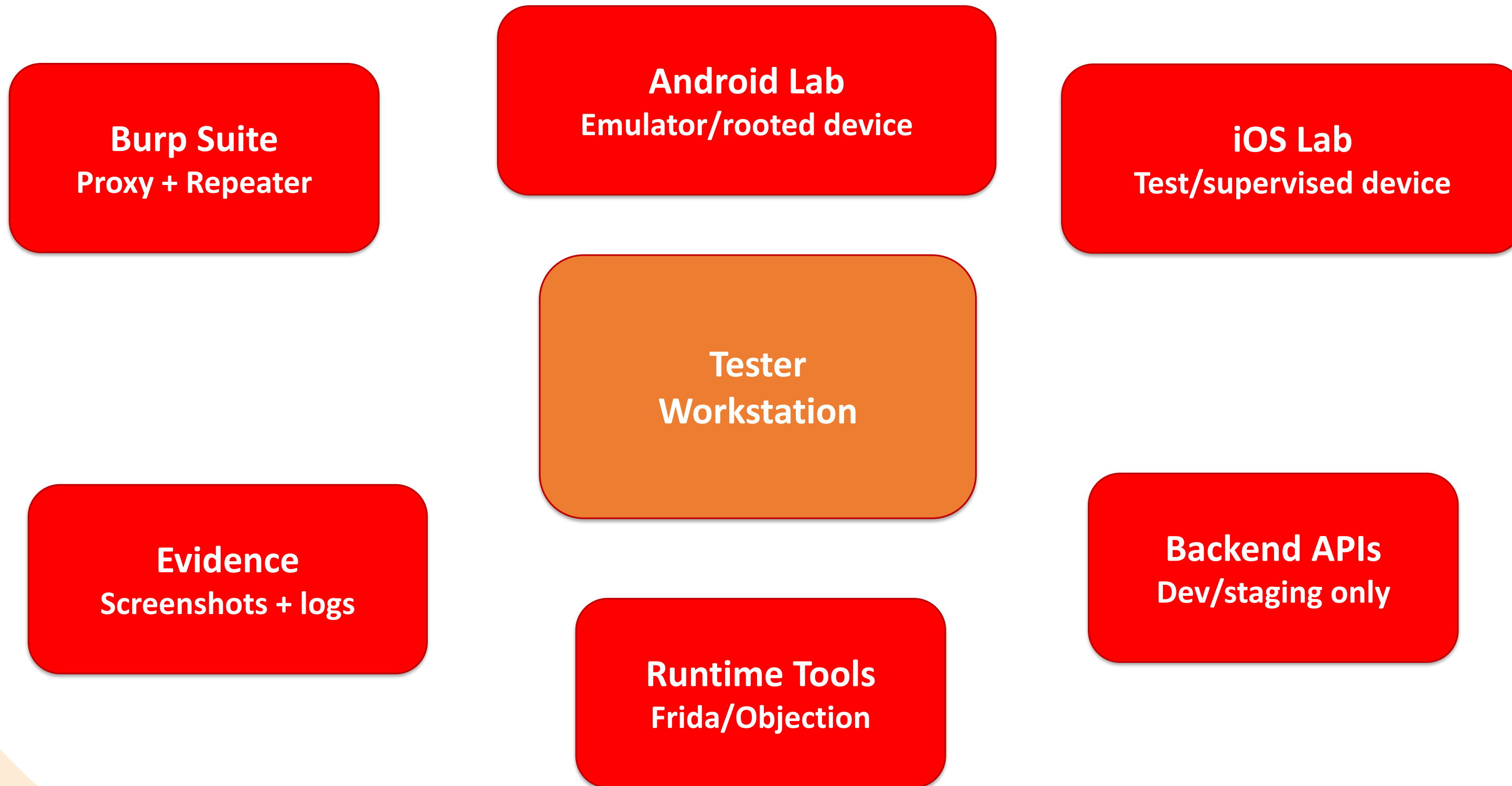


Mobile App Pentest Checklist

Area	What to Check	Evidence
Authentication	MFA, session expiry, token storage, device binding	Requests, token lifetime, screenshots
Authorization	Role checks, IDOR, object access through APIs	Original vs modified request/response
Data storage	PII, tokens, files, cache, logs, screenshots	File paths, screenshots, redacted samples
Transport	TLS, pinning, proxy behavior, sensitive data in URLs	Burp history, cert behavior, endpoints



Mobile Penetration Testing Lab Architecture



How WhatsApp Can Be Compromised

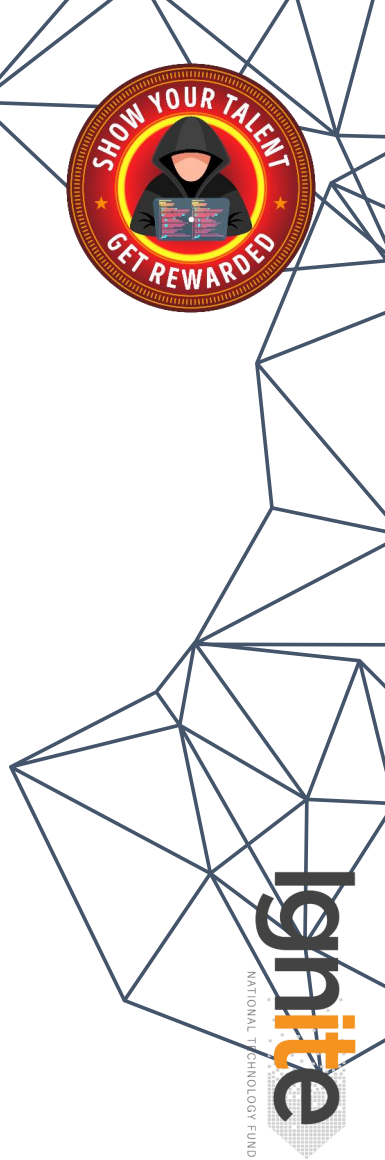
- End-to-end encryption protects message content in transit, but the account and device can still be targeted.
- Common compromise paths: stolen phone, weak screen lock, SIM swap/OTP theft, phishing, malicious apps or spyware.
- Linked-device abuse is common: an attacker gets a user to approve WhatsApp Web/Desktop pairing.
- Cloud backups, Google/Apple account takeover and unencrypted device backups can expose chat history.
- Group invites, impersonation and social engineering

Device
Stolen/unlocked
phone

Account
SIM/OTP/social
engineering

Backups
Cloud account
takeover

Linked Devices
Unauthorized
sessions



WhatsApp Risk Controls

- Use strong device lock, biometric unlock and short auto-lock timeout.
- Enable two-step verification PIN and protect the linked email/Apple/Google account with MFA.
- Review linked devices regularly and remove unknown sessions immediately.
- Avoid sharing sensitive data in personal groups; prefer approved channels.
- Restrict cloud backups for sensitive users or enforce encrypted backup policy where allowed.
- Treat unexpected QR pairing requests, APK files

**Two-Step
Verification PIN**

**Linked Devices
Review +
remove**

**Backups
Control policy**

**Official Channels
Use approved
apps**



Key Takeaways

- Mobile app penetration testing is not only the phone app; backend APIs, identity, storage and traffic are usually where impact appears.
- Android allows more flexible instrumentation; iOS usually needs stricter device, signing and jailbreak/supervision planning.
- WhatsApp content is encrypted in transit, but accounts can still be exposed through device compromise, backups, linked devices and social engineering.

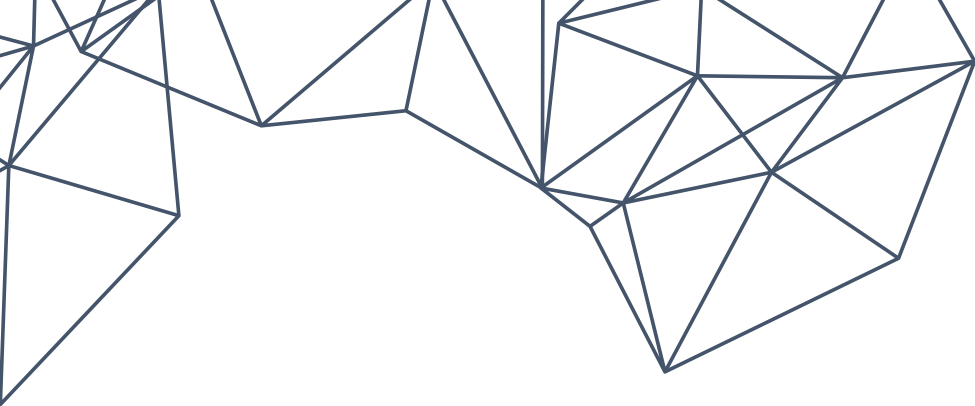
**Pentest
App + API +
device**

**WhatsApp
Account/device
risk**



Ignite
NATIONAL TECHNOLOGY FUND

TRILLIUM
INFORMATION SECURITY SYSTEMS



Thank You

